TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO SUPERIOR DE MONCLOVA

UNIDAD 6 · MÉTODOS DE BÚSQUEDA

TAREA 6.2

Práctica de programación y evaluación empírica

Implementación reproducible y banco de pruebas de búsquedas

Estudiante: Aldo Alberto Barajas Ancira

Matrícula: I24050397

Docente: Rubén Riojas

Área: Estructuras de Datos

Fecha: Noviembre de 2026

1. Objetivos y alcance

Implementar búsqueda secuencial, binaria iterativa y una tabla HASH desde cero; contrastar su corrección, registrar comparaciones e iteraciones y evaluar consultas exitosas y fallidas. El estudio incorpora carga de la tabla, colisiones controladas y tiempo de CPU como métrica complementaria.

2. Descripción de la implementación

Método	Implementación	Datos utilizados
Secuencial	Ciclo que inspecciona uno a uno elementos hasta coincidencia o fin.	Arreglo original desordenado.
Binaria	Intervalo de índices izquierdo/derecho y punto medio; 1 o 2 comparaciones de clave por paso.	Copia del mismo conjunto ordenada con sorted().
HASH	Cubetas con listas; índice h(k)=k mod m; encadenamiento separado, duplicados omitidos.	Mismas claves en tabla propia con m primo y α cercano a 0.75.

El proyecto no usa dict ni set para realizar búsquedas HASH. Sí usa herramientas estándar de Python para crear los datos y verificar propiedades; ello no reemplaza los algoritmos implementados.

3. Diseño del experimento

• Tamaños: n = 10,000, 50,000 y 100,000 enteros positivos únicos, seleccionados aleatoriamente del intervalo $[1, 10^9)$.

• Repeticiones independientes: tres semillas fijas (6051, 6052 y 6053) por tamaño, con la semilla combinada con n.

• Consultas: 100 claves presentes y 100 ausentes por semilla; las ausentes son negativas y ninguna puede pertenecer al conjunto positivo. En total: 300 consultas exitosas y 300 fallidas por n y método.

• Equidad: cada método responde a las mismas claves; las colecciones tienen el mismo contenido, aunque binaria requiere una representación ordenada.

• Verificación: se comprueba que toda búsqueda presente devuelve verdadero y toda ausente devuelve falso; pruebas automáticas adicionales cubren vacíos, negativos, duplicados y colisiones.

3.1. Definiciones operacionales

Una comparación es una evaluación explícita entre la clave candidata y el objetivo (igualdad == o relación <). Una iteración es una visita a un candidato, ya sea elemento del arreglo o entrada de una cadena. Binaria puede efectuar dos comparaciones en una sola iteración. HASH puede requerir cero comparaciones en una cubeta vacía, pero aun así realiza un acceso a cubeta; este se registra por separado.

El tiempo de CPU se mide con process_time_ns() por lote de consultas y se divide entre su número. La ordenación previa y la construcción de HASH se cronometran aparte; no se suman al costo de consulta. Los tiempos son observaciones del entorno de ejecución de esta práctica y no garantías de rendimiento en otro equipo.

4. Resultados principales: comparaciones e iteraciones

Las cifras siguientes son promedios por consulta agregados de tres semillas, con 300 consultas por condición y tamaño en cada técnica. Los CSV conservan tanto los totales como las observaciones individuales.

4.1. Consultas exitosa

N	Algoritmo	Comparac.	Iteraciones	Accesos cubeta	CPU/consulta (µs)
10,000	Secuencial	4,953.49	4,953.49	0.00	229.11
10,000	Binaria	23.59	12.29	0.00	2.92
10,000	Hash encadenada	1.40	1.40	1.00	1.11
50,000	Secuencial	25,967.12	25,967.12	0.00	1,166.32
50,000	Binaria	28.19	14.60	0.00	3.24
50,000	Hash encadenada	1.40	1.40	1.00	1.05
100,000	Secuencial	45,710.47	45,710.47	0.00	2,395.72
100,000	Binaria	30.52	15.76	0.00	4.21
100,000	Hash encadenada	1.36	1.36	1.00	1.09

4.2. Consultas fallida

N	Algoritmo	Comparac.	Iteraciones	Accesos cubeta	CPU/consulta (µs)
10,000	Secuencial	10,000.00	10,000.00	0.00	465.97
10,000	Binaria	26.00	13.00	0.00	2.15
10,000	Hash encadenada	0.72	0.72	1.00	1.09
50,000	Secuencial	50,000.00	50,000.00	0.00	2,185.29
50,000	Binaria	30.00	15.00	0.00	2.21
50,000	Hash encadenada	0.80	0.80	1.00	0.84
100,000	Secuencial	100,000.00	100,000.00	0.00	5,353.29
100,000	Binaria	32.00	16.00	0.00	2.34
100,000	Hash encadenada	0.78	0.78	1.00	0.99

Interpretación: el número de comparaciones de HASH puede ser inferior a 1 en búsquedas fallidas porque muchas cubetas están vacías. Esto no significa que una consulta HASH carezca de costo: la dispersión y el acceso a la cubeta siguen ocurriendo. En binaria, las comparaciones superan a las iteraciones porque el código distingue igualdad y orden.

5. Gráficas comparativas

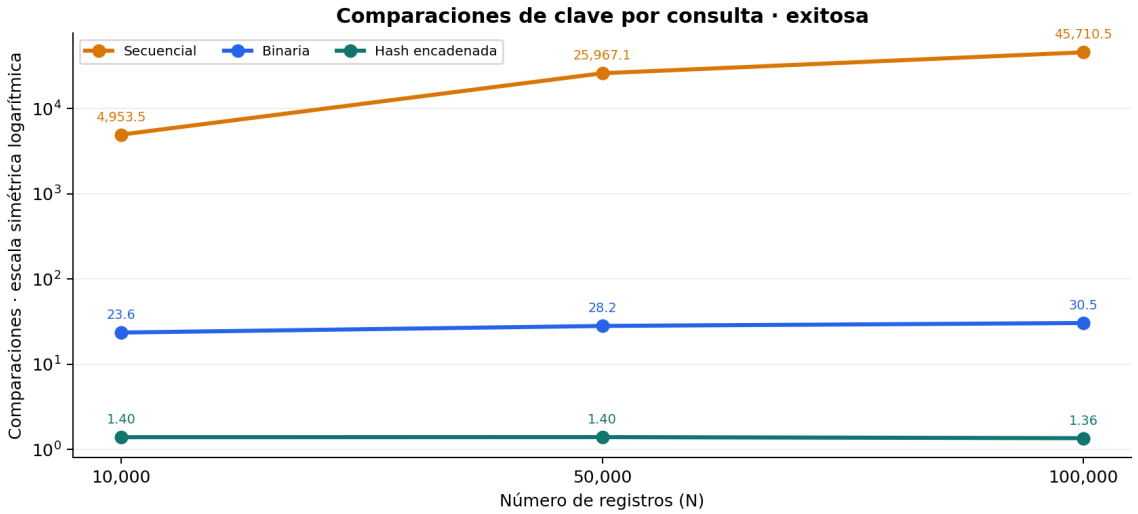


Figura 1. Comparaciones por consulta exitosa; eje vertical con escala simétrica logarítmica.

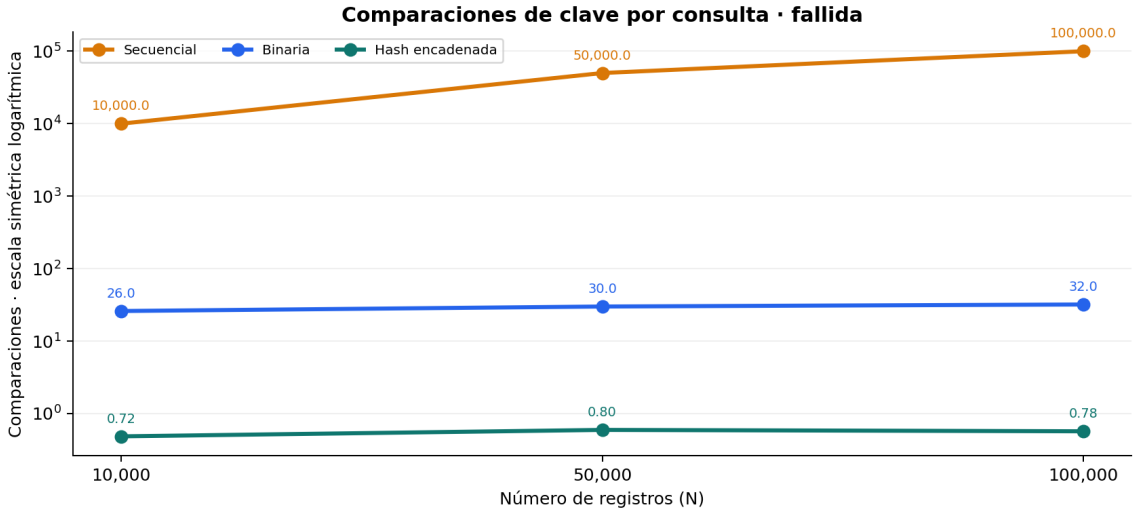


Figura 2. Comparaciones por consulta fallida; cifras exactas en las tablas de la sección 4.

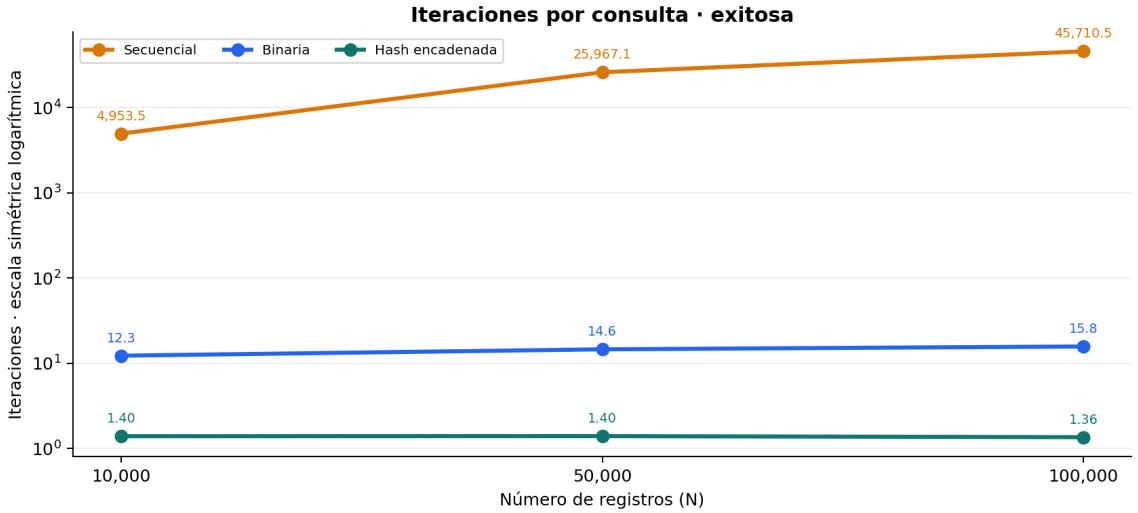


Figura 3. Iteraciones en búsquedas exitosas para los tres tamaños.

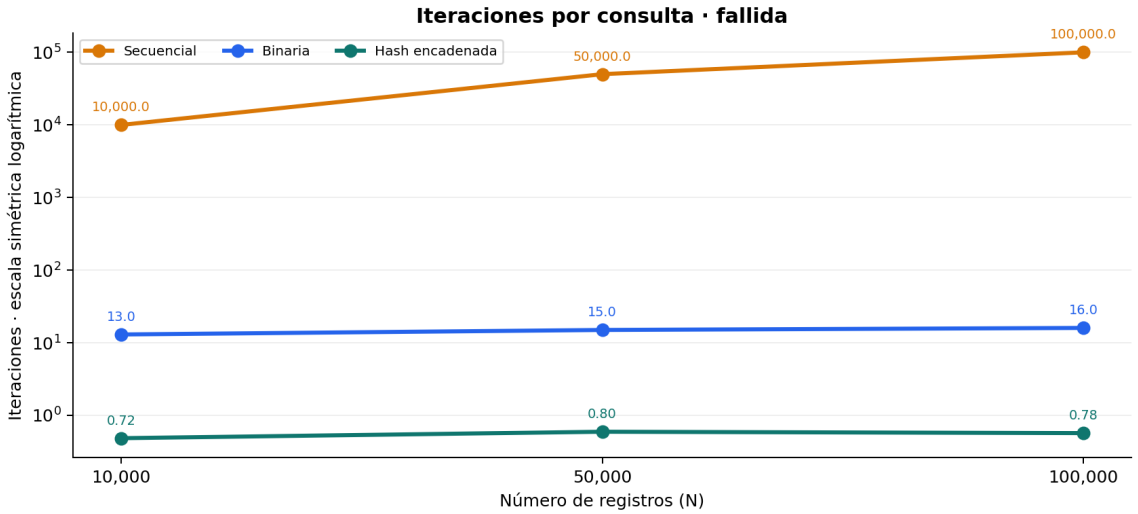


Figura 4. Iteraciones en búsquedas fallidas para los tres tamaños.

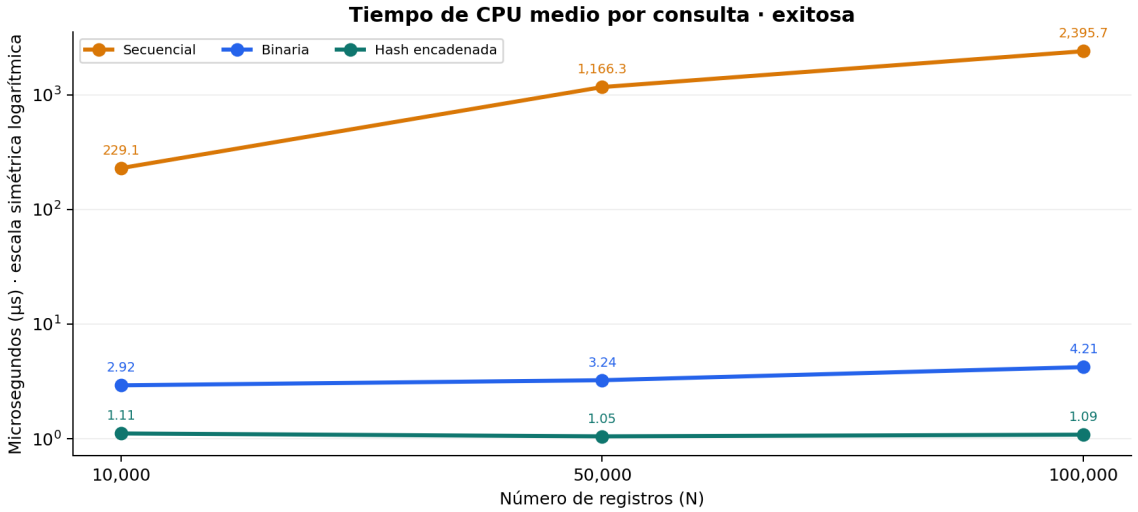


Figura 5. Tiempo de CPU promedio en búsquedas exitosas; no incluye preprocesamiento.

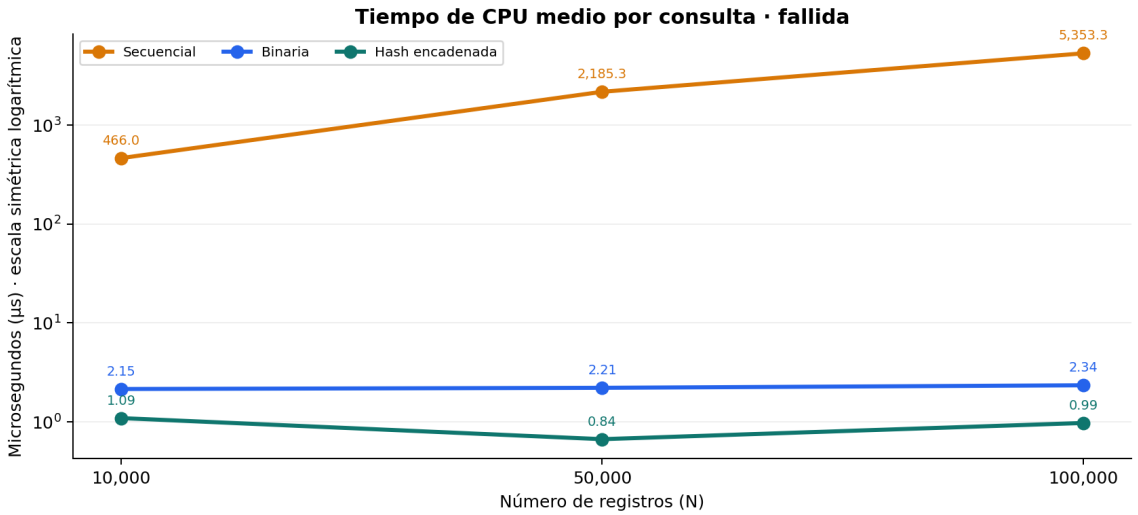


Figura 6. Tiempo de CPU promedio en búsquedas fallidas; dependiente del entorno.

6. Costo de preparar las estructuras

N	Ordenar CPU ms	Crear HASH CPU ms	Cubetas HASH	Colisiones inserción (prom.)	Cadena máx.
10,000	1.23	24.67	13,337	3,007	7
50,000	7.06	14.48	66,683	14,811	7
100,000	16.24	97.84	133,337	29,565	8

El tiempo de construcción HASH incluye reservar cubetas, insertar claves y verificar duplicados por cadena (no incluye calcular la capacidad); el tiempo de ordenación corresponde a sorted(datos). La elección para una sola búsqueda podría diferir de la elección para miles de consultas, pues la preparación no es gratuita.

7. Colisiones y factor de carga

Se mantuvieron 50,000 claves y se varió m para evaluar α = n/m. Las entradas son iguales en todos los ensayos; se realizaron 200 consultas presentes y 200 ausentes por nivel de carga. A diferencia del sondeo abierto, el encadenamiento separado admite α mayor que 1.

Carga objetivo	α real	Consulta	Comparac. promedio	Colisiones insertadas	Cadena máx.
0.5	0.5	Exitosa	1.29	10791	5
0.5	0.5	Fallida	0.58	10791	5
0.75	0.7498	Exitosa	1.265	14733	7
0.75	0.7498	Fallida	0.685	14733	7
1.0	0.9996	Exitosa	1.545	18271	7

Carga objetivo	α real	Consulta	Comparac. promedio	Colisiones insertadas	Cadena máx.
1.0	0.9996	Fallida	1.035	18271	7
1.5	1.4996	Exitosa	1.77	24134	9
1.5	1.4996	Fallida	1.52	24134	9
2.0	1.999	Exitosa	2.115	28398	11
2.0	1.999	Fallida	1.845	28398	11

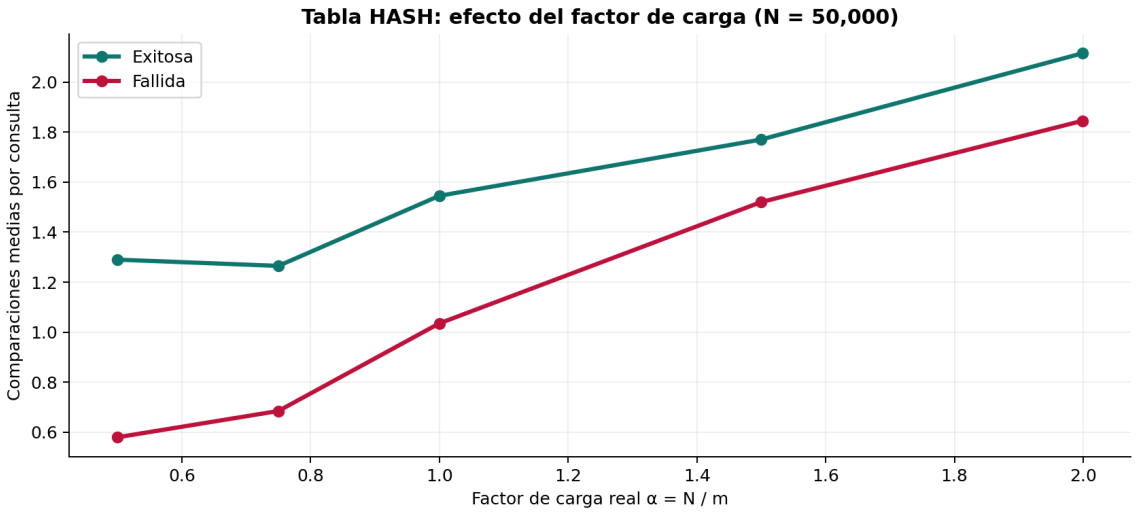


Figura 7. Con más carga y claves distribuidas aleatoriamente, las cadenas tienden a alargarse.

7.1. Ensayo patológico reproducible

Se introdujeron 500 claves distintas de la forma 7 + i × 1009 (i de 0 a 499) en una tabla de 1009 cubetas. Todas tienen índice HASH 7. Así se construyó intencionalmente una cadena de longitud 500 que demuestra el peor caso.

Consulta	Índice	¿Existe?	Comparaciones	Iteraciones
Primera insertada	7	Sí	1	1
Última insertada	7	Sí	500	500
Ausente mismo índice	7	No	500	500
Ausente cubeta vacía	8	No	0	0

8. Discusión de los resultados

Con n = 100,000, las búsquedas fallidas registraron 100,000 comparaciones secuenciales frente a 32 en binaria y 0.78 en HASH. La búsqueda secuencial examina todo el conjunto para confirmar la ausencia, mientras que la binaria descarta mitades y HASH revisa una sola cubeta, aunque debe primero construir su índice.

En búsquedas exitosas, la posición de la clave determina cuántos elementos recorre la secuencial. La binaria tiene un crecimiento lento, compatible con O(log n). HASH conserva pocas visitas con carga limitada y dispersión razonablemente uniforme, pero el caso forzado exhibe 500 comparaciones en una sola cubeta, confirmando que su peor caso no es O(1).

8.1. Limitaciones y validez

• Los datos y consultas son aleatorios reproducibles, pero no representan todos los patrones reales ni claves intencionalmente maliciosas. Se añadió por separado un escenario de colisiones forzadas.

• Las mediciones de CPU en Python son sensibles al intérprete, sistema operativo, procesador y ejecución concurrente. Las comparaciones e iteraciones son más transportables para análisis algorítmico.

• La búsqueda secuencial usa una lista sin ordenar; binaria utiliza copia ordenada; HASH necesita reservar y construir su tabla. No se interpretan los tiempos de consulta como un costo total de extremo a extremo.

• La función modular h(k)=k mod m se usa con fines didácticos; no es una política de seguridad ante entradas adversarias ni sustituye un diseño HASH robusto para producción.

9. Conclusiones empíricas

El experimento respalda el patrón teórico: secuencial crece aproximadamente de forma lineal, binaria realiza unas pocas decenas de comparaciones incluso con 100,000 registros y HASH suele examinar pocas claves mientras las colisiones estén distribuidas. Sin embargo, HASH demanda memoria y preparación; el caso adversarial muestra que su eficacia depende crucialmente de la función de dispersión y de cómo se resuelven las colisiones. Para una solución real deben valorarse cantidad de consultas, frecuencia de actualización, necesidad de orden y patrón de claves.

10. Instrucciones de reproducción

```
python codigo/demo_interactiva.py

python -m unittest discover -s codigo -p "prueba_*.py" -v

python codigo/experimento.py --salida datos

python codigo/generar_graficas.py --datos datos --salida graficas
```

Las primeras tres órdenes funcionan con Python 3.10+ y biblioteca estándar. Para regenerar las gráficas se necesita instalar matplotlib. Los CSV incluyen datos por semilla, resumen, preparación, estudio de cargas y colisiones forzadas.

Referencias consultadas

[1] NIST, Dictionary of Algorithms and Data Structures. Sequential search. https://xlinux.nist.gov/dads/HTML/sequentialSearch.html

[2] CP-Algorithms. Binary search. https://cp-algorithms.com/num_methods/binary_search.html

[3] Sedgewick & Wayne, Princeton University. Hash tables (Algorithms, 4th Edition). https://algs4.cs.princeton.edu/34hash/

[4] OpenDSA / Virginia Tech. Open Hashing: separate chaining. https://opendsa-server.cs.vt.edu/ODSA/Books/pubbook/odsa-all/fall-2019/Public_Instance/html/OpenHash.html

[5] OpenDSA. Collision Resolution / Open Addressing. https://opendsa-server.cs.vt.edu/ODSA/StandaloneModules/help/20221201151914/html/HashCSimple.html

Consulta de fuentes: noviembre de 2026. Las complejidades son asintóticas y suponen el modelo de memoria indicado.